Outline

6.1 Introduction
6.2 Arrays
6.3 Defining Arrays
6.4 Array Examples
- 6.4.1 Defining an Array and Using a Loop to Set the Array's Element Values
- 6.4.2 Initializing an Array in a Definition with an Initializer List
- 6.4.3 Specifying an Array's Size with a Symbolic Constant and Initializing Array Elements with Calculations
- 6.4.4 Summing the Elements of an Array
- 6.4.5 Using Arrays to Summarize Survey Results
- 6.4.6 Graphing Array Element Values with Bar Charts
- 6.4.7 Rolling a Die 60,000,000 Times and Summarizing the Results in an Array

6.5 Using Character Arrays to Store and Manipulate Strings
- 6.5.1 Initializing a Character Array with a String
- 6.5.2 Initializing a Character Array with an Initializer List of Characters
- 6.5.3 Accessing the Characters in a String
- 6.5.4 Inputting into a Character Array
- 6.5.5 Outputting a Character Array That Represents a String
- 6.5.6 Demonstrating Character Arrays

6.6 Static Local Arrays and Automatic Local Arrays
6.7 Passing Arrays to Functions
6.8 Sorting Arrays
6.9 Intro to Data Science Case Study: Survey Data Analysis
6.10 Searching Arrays
- 6.10.1 Searching an Array with Linear Search
- 6.10.2 Searching an Array with Binary Search

6.11 Multidimensional Arrays
- 6.11.1 Illustrating a Two-Dimensional Array
- 6.11.2 Initializing a Double-Subscripted Array
- 6.11.3 Setting the Elements in One Row
- 6.11.4 Totaling the Elements in a Two-Dimensional Array
- 6.11.5 Two-Dimensional Array Manipulations

6.12 Variable-Length Arrays
6.13 Secure C Programming

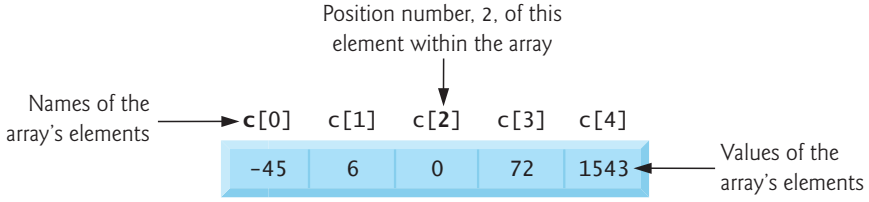
Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises* | *Recursion Exercises*

6.1 Introduction

This chapter introduces data structures. Arrays are data structures consisting of related data items of the same type. Chapter 10 discusses C's notion of `struct`—a data structure consisting of related data items of possibly different types. Arrays and `structs` are "static" entities in that they remain the same size throughout their lifetimes.

6.2 Arrays

An array is a group of elements of the same type stored contiguously in memory. The following diagram shows an integer array called `c`, containing five elements:



To refer to a particular location or element in the array, we specify the array's name, followed by the element's **position number** in square brackets (`[]`). The first element is located at position number `0` (zero). The position number is called the element's **subscript** (or **index**). A subscript must be a non-negative integer or integer expression.

Let's examine the array in the previous diagram more closely. The array's **name** is `c`. The **value** of `c[0]` is `-45`, `c[2]` is `0` and `c[4]` is `1543`. A subscripted array name is an *lvalue* that can be used on the left side of an assignment. So, the statement:

```
c[2] = 1000;
```

replaces `c[2]`'s current value (`0`) with the value `1000`. To print the sum of the values in array `c`'s first three elements, we'd write:

```
printf("%d", c[0] + c[1] + c[2]);
```

To divide the value of element `3` of array `c` by `2` and assign the result to the variable `x`, write:

```
x = c[3] / 2;
```

The brackets that enclose an array's subscript are an operator with the highest level of precedence. The following table shows the precedence and grouping of the operators introduced to this point in the text:

Operators	Grouping	Type
`[]` `()` `++`*(postfix)* `--`*(postfix)*	left to right	highest
`+` `-` `!` `++`*(prefix)* `--`*(prefix)* `(`*type*`)`	right to left	unary
`*` `/` `%`	left to right	multiplicative
`+` `-`	left to right	additive
`<` `<=` `>` `>=`	left to right	relational
`==` `!=`	left to right	equality
`&&`	left to right	logical AND
`||`	left to right	logical OR
`?:`	right to left	conditional
`=` `+=` `-=` `*=` `/=` `%=`	right to left	assignment
`,`	left to right	comma

Self Check

1 ***(Multiple Choice)*** Which of the following statements is *false*?

a) Any array element may be referred to by giving the array's name followed by the element's position number in square brackets (`[]`).

b) Every array's first element has position number `1`.

c) An array name, like other identifiers, can contain only letters, digits and underscores and cannot begin with a digit.

d) The position number in square brackets is called the element's subscript (or index), which must be an integer or an integer expression.

Answer: b) is *false*. Actually, every array's first element has position number `0`.

2 *(Code)* Write a statement that displays the `int` product of the values contained in the first four elements of `int` array `grades`.

Answer: `printf("%d", grades[0] * grades[1] * grades[2] * grades[3]);`

6.3 Defining Arrays

When you define an array, you specify its element type and number of elements so the compiler may reserve the appropriate amount of memory. The following definition reserves five elements for integer array `c`, which has subscripts in the range 0–4.

```
int c[5];
```

The definitions

```
int b[100];
int x[27];
```

reserve 100 elements for integer array `b` and 27 elements for integer array `x`. These arrays have subscripts in the ranges 0–99 and 0–26, respectively.

A `char` array can store a character string. Character strings and their similarity to arrays are discussed in Chapter 8. The relationship between pointers and arrays is discussed in Chapter 7.

Self Check

1 *(Multiple Choice)* Which of the following statements is *false*?

a) When creating an array, you specify an array's element type and number of elements so the compiler may reserve the appropriate amount of memory.

b) The following definition reserves space for the `double` array `temperatures`, which has index numbers in the range `0`–`6`:

```
double temperatures[7];
```

c) The following definitions reserve 50 elements for `float` array `b` and `19` elements for `float` array `x`:

```
float b[50];
float x[19];
```

d) An array of type `string` can store a character string.

Answer: d) is *false*. Actually, an array of type `char` can store a character string. C does not have a `string` type.

6.4 Array Examples

This section presents several examples demonstrating how to define and initialize arrays and how to perform many common array manipulations.

6.4.1 Defining an Array and Using a Loop to Set the Array's Element Values

Like any other local variable, uninitialized array elements contain "garbage" values. Figure 6.1 uses `for` statements to set five-element integer array `n`'s elements to zeros (lines 10–12) and print the array in tabular format (lines 17–19). The first `printf` statement (line 14) displays the column heads for the two columns printed in the subsequent `for` statement.

```
1  // fig06_01.c
2  // Initializing the elements of an array to zeros.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int n[5]; // n is an array of five integers
8
9     // set elements of array n to 0
10    for (size_t i = 0; i < 5; ++i) {
11       n[i] = 0; // set element at location i to 0
12    }
13
14    printf("%s%8s\n", "Element", "Value");
15
16    // output contents of array n in tabular format
17    for (size_t i = 0; i < 5; ++i) {
18       printf("%7zu%8d\n", i, n[i]);
19    }
20 }
```

```
Element   Value
      0       0
      1       0
      2       0
      3       0
      4       0
```

Fig. 6.1 | Initializing the elements of an array to zeros.

The counter-control variable `i`'s type is `size_t` in each `for` statement (lines 10 and 17). The C standard says `size_t` represents an unsigned integral type and is recommended for any variable representing an array's size or subscripts. Type `size_t` is defined in header `<stddef.h>`, which is often included by other headers (such as `<stdio.h>`).[1] The **conversion specification `%zu`** is used to display `size_t` values.

1. If you attempt to compile Fig. 6.1 and receive errors, include `<stddef.h>` in your program.

6.4.2 Initializing an Array in a Definition with an Initializer List

You can initialize an array's elements when defining the array by providing a comma-separated list of **array initializers** in braces, `{}`. Figure 6.2 initializes an integer array with five values (line 7) and prints it in a tabular format.

```
1  // fig06_02.c
2  // Initializing the elements of an array with an initializer list.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int n[5] = {32, 27, 64, 18, 95}; // initialize n with initializer list
8
9     printf("%s%8s\n", "Element", "Value");
10
11    // output contents of array in tabular format
12    for (size_t i = 0; i < 5; ++i) {
13       printf("%7zu%8d\n", i, n[i]);
14    }
15 }
```

```
Element   Value
      0      32
      1      27
      2      64
      3      18
      4      95
```

Fig. 6.2 | Initializing the elements of an array with an initializer list.

If there are fewer initializers than array elements, the remaining elements are initialized to `0`. For example, Fig. 6.1 could have initialized array `n`'s elements to zero as follows:

```
int n[5] = {0}; // initializes entire array to zeros
```

This explicitly initializes `n[0]` to `0` and implicitly initializes the remaining elements to `0`. It's a compilation error if you provide more initializers in an array initializer list than elements in the array. For example, the following array definition produces a compilation error because there are four initializers for only three elements:

ERR

```
int n[3] = {32, 27, 64, 18};
```

The following definition creates a five-element array initialized with the values `1`–`5`:

```
int n[] = {1, 2, 3, 4, 5};
```

When you omit the array size, the compiler calculates the array's number of elements from the number initializers.

6.4.3 Specifying an Array's Size with a Symbolic Constant and Initializing Array Elements with Calculations

Figure 6.3 initializes five-element array `s` with the values `2`, `4`, `6`, `8` and `10`, then prints the array in tabular format. To generate the values, we multiply the loop counter by `2` and add `2`.

```
1  // fig06_03.c
2  // Initializing the elements of array s to the even integers from 2 to 10.
3  #include <stdio.h>
4  #define SIZE 5 // maximum size of array
5
6  // function main begins program execution
7  int main(void) {
8     // symbolic constant SIZE can be used to specify array size
9     int s[SIZE] = {0}; // array s has SIZE elements
10
11    for (size_t j = 0; j < SIZE; ++j) { // set the values
12       s[j] = 2 + 2 * j;
13    }
14
15    printf("%s%8s\n", "Element", "Value");
16
17    // output contents of array s in tabular format
18    for (size_t j = 0; j < SIZE; ++j) {
19       printf("%7zu%8d\n", j, s[j]);
20    }
21 }
```

```
Element   Value
      0       2
      1       4
      2       6
      3       8
      4      10
```

Fig. 6.3 | Initializing the elements of array `s` to the even integers from 2 to 10.

Line 4 uses the **`#define` preprocessor directive**

```
#define SIZE 5
```

to create the **symbolic constant** `SIZE` with the value `5`. A symbolic constant is an identifier that the C preprocessor replaces with **replacement text** before the program is compiled. In this program, the preprocessor replaces all `SIZE` occurrences with `5`.

Using symbolic constants to specify array sizes makes programs easier to read and modify. In Fig. 6.3, for example, we could have the first `for` loop (line 11) fill a 1000-element array simply by changing `SIZE`'s value in the `#define` directive from `5` to `1000`. Without the symbolic constant, we'd have to change the program in lines 9, 11 and 18. As programs get larger, this technique becomes more useful for writing clear, easy-to-read, maintainable programs. A symbolic constant (like `SIZE`) is easier to understand than the numeric value `5`, which could have different meanings throughout the code.

Do not terminate the `#define` preprocessor directives with semicolons. If you do that in line 4, the preprocessor replaces all occurrences of `SIZE` with the text `"5;"`. This may lead to syntax errors at compile time or logic errors at execution time. Remember that the preprocessor is not the C compiler.

ERR

Assigning a value to a symbolic constant in an executable statement is a compilation error—symbolic constants are not variables. By convention, use only uppercase letters for symbolic constant names, so they stand out in a program. This also reminds you that symbolic constants are not variables.

ERR

6.4.4 Summing the Elements of an Array

Figure 6.4 sums the values contained in the five-element integer array `a`. The `for` statement's body (line 14) does the totaling.

```
1  // fig06_04.c
2  // Computing the sum of the elements of an array.
3  #include <stdio.h>
4  #define SIZE 5
5
6  // function main begins program execution
7  int main(void) {
8     // use an initializer list to initialize the array
9     int a[SIZE] = {1, 2, 3, 4, 5};
10    int total = 0; // sum of array
11
12    // sum contents of array a
13    for (size_t i = 0; i < SIZE; ++i) {
14       total += a[i];
15    }
16
17    printf("The total of a's values is %d\n", total);
18 }
```

```
The total of a's values is 15
```

Fig. 6.4 | Computing the sum of the elements of an array.

6.4.5 Using Arrays to Summarize Survey Results

Our next example uses arrays to summarize the results of data collected in a survey. Consider the problem statement:

> *Twenty students were asked to rate the quality of the food in the student cafeteria on a scale of 1 to 5 (1 means awful, and 5 means excellent). Place the 20 responses in an integer array and summarize the results of the poll.*

Figure 6.5 is a typical array application. We wish to summarize the number of responses of each type. The 20-element array `responses` (lines 10–11) contains the students' responses. We use a 6-element array `frequency` (line 14) to count each response's number of occurrences. We ignore `frequency[0]` because it's logical to

have response 1 increment `frequency[1]` rather than `frequency[0]`. This allows us to use each response directly as the subscript in the `frequency` array. You should strive for program clarity. Sometimes it may be worthwhile to trade off the most efficient use of memory or processor time in favor of writing clearer programs. Sometimes performance considerations far outweigh clarity considerations. PERF

```
1  // fig06_05.c
2  // Analyzing a student poll.
3  #include <stdio.h>
4  #define RESPONSES_SIZE 20 // define array sizes
5  #define FREQUENCY_SIZE 6
6
7  // function main begins program execution
8  int main(void) {
9     // place the survey responses in the responses array
10    int responses[RESPONSES_SIZE] =
11       {1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};
12
13    // initialize frequency counters to 0
14    int frequency[FREQUENCY_SIZE] = {0};
15
16    // for each answer, select the value of an element of the array
17    // responses and use that value as a subscript into the array
18    // frequency to determine the element to increment
19    for (size_t answer = 0; answer < RESPONSES_SIZE; ++answer) {
20       ++frequency[responses[answer]];
21    }
22
23    // display results
24    printf("%s%12s\n", "Rating", "Frequency");
25
26    // output the frequencies in a tabular format
27    for (size_t rating = 1; rating < FREQUENCY_SIZE; ++rating) {
28       printf("%6zu%12d\n", rating, frequency[rating]);
29    }
30 }
```

```
Rating   Frequency
     1           3
     2           5
     3           7
     4           2
     5           3
```

Fig. 6.5 | Analyzing a student poll.

How the `frequency` Counters Are Incremented

The `for` loop (lines 19–21) takes each response from `responses` and increments one of the five `frequency` array counters—`frequency[1]` to `frequency[5]`. The key statement in the loop is line 20:

```
++frequency[responses[answer]];
```

which increments the appropriate `frequency` counter, based on the value of the expression `responses[answer]`. When the counter variable `answer` is `0`, `responses[answer]` is `1`, so `++frequency[responses[answer]];` is interpreted as

```
++frequency[1];
```

which increments `frequency[1]`. When `answer` is `1`, the value of `responses[answer]` is `2`, so `++frequency[responses[answer]];` is interpreted as

```
++frequency[2];
```

which increments `frequency[2]`. When `answer` is `2`, the value of `responses[answer]` is `5`, so `++frequency[responses[answer]];` is interpreted as

```
++frequency[5];
```

which increments `frequency[5]`, and so on.

Invalid Survey Responses

Regardless of the number of survey responses processed, only a 6-element `frequency` array is required (ignoring element zero) to summarize the results. But what if the data contained an invalid value such as 13? In this case, the program would attempt to add `1` to `frequency[13]`, which is outside the array's bounds. C has no array bounds checking to prevent a program from referring to an element that does not exist. So, an executing program can "walk off" either end of an array without warning—a security problem we discuss in Section 6.13. Programs should validate that all input values are correct to prevent erroneous information from affecting a program's calculations.

SEC

Validate Array Subscripts

ERR

Referring to an element outside the array bounds is a logic error. When looping through an array, the array subscript should never go below 0 and should always be less than the total number of array elements—the array's size minus one. You should ensure that all array references remain within the bounds of the array.

6.4.6 Graphing Array Element Values with Bar Charts

Our next example (Fig. 6.6) reads numbers from an array and graphs the information in a bar chart. We display each number followed by a bar consisting of that many asterisks. The nested `for` statement (lines 17–19) displays the bars by iterating `n[i]` times and displaying one asterisk per iteration. Line 21 ends each bar.

```
1  // fig06_06.c
2  // Displaying a bar chart.
3  #include <stdio.h>
4  #define SIZE 5
5
6  // function main begins program execution
7  int main(void) {
```

Fig. 6.6 | Displaying a bar chart. (Part 1 of 2.)

```
    // use initializer list to initialize array n
    int n[SIZE] = {19, 3, 15, 7, 11};

    printf("%s%13s%17s\n", "Element", "Value", "Bar Chart");

    // for each element of array n, output a bar of the bar chart
    for (size_t i = 0; i < SIZE; ++i) {
       printf("%7zu%13d%8s", i, n[i], "");

       for (int j = 1; j <= n[i]; ++j) { // print one bar
          printf("%c", '*');
       }

       puts(""); // end a bar with a newline
    }
 }
```

```
Element        Value        Bar Chart
      0           19        *******************
      1            3        ***
      2           15        ***************
      3            7        *******
      4           11        ***********
```

Fig. 6.6 | Displaying a bar chart. (Part 2 of 2.)

6.4.7 Rolling a Die 60,000,000 Times and Summarizing the Results in an Array

In Chapter 5, we stated that we'd show a more elegant way to write Fig. 5.5's dice-rolling program. Recall that the program rolled a single six-sided die 60,000,000 times and displayed the face counts. Figure 6.7 is an array version of Fig. 5.5. Line 17 replaces Fig. 5.5's entire 20-line `switch` statement. Once again, we use a `frequency` array in which we ignore element `0`, so we can use the face values as subscripts into the array.

```
1  // fig06_07.c
2  // Roll a six-sided die 60,000,000 times
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <time.h>
6  #define SIZE 7
7
8  // function main begins program execution
9  int main(void) {
10    srand(time(NULL)); // seed random number generator
11
12    int frequency[SIZE] = {0}; // initialize all frequency counts to 0
13
```

Fig. 6.7 | Roll a six-sided die 60,000,000 times. (Part 1 of 2.)

```
    // roll die 60,000,000 times
    for (int roll = 1; roll <= 60000000; ++roll) {
       size_t face = 1 + rand() % 6;
       ++frequency[face]; // replaces entire switch of Fig. 5.5
    }

    printf("%s%17s\n", "Face", "Frequency");

    // output frequency elements 1-6 in tabular format
    for (size_t face = 1; face < SIZE; ++face) {
       printf("%4zu%17d\n", face, frequency[face]);
    }
 }
```

```
Face        Frequency
   1          9997167
   2         10003506
   3         10001940
   4          9995833
   5         10000843
   6         10000711
```

Fig. 6.7 | Roll a six-sided die 60,000,000 times. (Part 2 of 2.)

Self Check

1 *(Code)* Rewrite the following code segment to define seven-element `double` array `m` and initialize each of its elements to `10`:

```
int n[5]; // n is an array of five integers

// set elements of array n to 0
for (size_t i = 0; i < 5; ++i) {
   n[i] = 0; // set element at location i to 0
}
```

Answer:

```
double m[7]; // m is an array of 7 doubles

// set elements of array m to 10.0
for (size_t i = 0; i < 7; ++i) {
   m[i] = 10.0; // set element at location i to 10.0
}
```

2 *(Multiple Choice)* Which of the following statements a), b) or c) is *true*?

a) For an `int` array, if you provide fewer initializers than there are elements in the array, the remaining elements are initialized to `0`.

b) It's a syntax error to provide more initializers in an array initializer list than there are array elements—for example, `int n[3] = {32, 27, 64, 18};` is a syntax error, because there are four initializers but only three array elements.

c) If the array size is omitted from a definition with an initializer list, the compiler determines the number of elements based on the number of elements in the initializer list. So, the following creates the three-element `int` array `s`:

```
int s[] = {10, 20, 30};
```

d) All of the above statements are *true*.

Answer: d.

6.5 Using Character Arrays to Store and Manipulate Strings

Arrays can hold data of any type, though all the elements of a given array must have the same type. We now discuss storing strings in character arrays. So far, the only string-processing capability we have is outputting a string with `printf`. A string such as `"hello"` is really an array of individual characters, plus one more thing.

6.5.1 Initializing a Character Array with a String

Character arrays have several unique features. A character array can be initialized using a string literal. For example,

```
char string1[] = "first";
```

initializes the elements of array `string1` to the individual characters in the string literal `"first"`. In this case, the compiler determined array `string1`'s size based on the string's length. The string `"first"` contains five characters *plus a string-terminating null character*. So, `string1` actually contains six elements. The escape sequence representing the null character is `'\0'`. All strings end with this character. A character array representing a string should always be defined large enough to hold the string's number of characters and the terminating null character.

6.5.2 Initializing a Character Array with an Initializer List of Characters

Character arrays also can be initialized with individual character constants in an initializer list, but this can be tedious. The preceding definition is equivalent to

```
char string1[] = {'f', 'i', 'r', 's', 't', '\0'};
```

6.5.3 Accessing the Characters in a String

You can access a string's individual characters directly using array subscript notation. So, `string1[0]` is the character `'f'`, `string1[3]` is `'s'` and `string1[5]` is `'\0'`.

6.5.4 Inputting into a Character Array

The following definition creates a character array capable of storing a string of *at most 19 characters* and a *terminating null character*:

```
char string2[20];
```

The statement

```
scanf("%19s", string2);
```

reads a string from the keyboard into `string2`. You pass the array name to `scanf` without the `&` used with non-string variables. The `&` is normally used to provide `scanf` with a variable's *location* in memory so that a value can be stored there. In Section 6.7, when we discuss passing arrays to functions, we'll discuss why the `&` is not necessary for array names.

It's your responsibility to ensure that the array into which the string is read is capable of holding any string that the user types at the keyboard. Function `scanf` does *not* check how large the array is. It will read characters until a *space*, *tab*, *newline* or *end-of-file indicator* is encountered. The string `string2` should be no longer than 19 characters to leave room for the terminating null character. If the user types 20 or more characters, your program may crash or create a security vulnerability called a buffer overflow. For this reason, we used the conversion specification `%19s`. This tells `scanf` to read a maximum of 19 characters, preventing it from writing characters into memory beyond the end of `string2`. (In Section 6.13, we revisit the potential security issue raised by inputting into a character array and discuss the C standard's `scanf_s` function.)

SEC

6.5.5 Outputting a Character Array That Represents a String

A character array representing a string can be output with `printf` using the `%s` conversion specification. For example, you can print the character array `string2` with

```
printf("%s\n", string2);
```

Like `scanf`, `printf` does not check how large the character array is. It displays the string's characters until it encounters a terminating null character. [Consider what would print if, for some reason, the terminating null character were missing.]

6.5.6 Demonstrating Character Arrays

Figure 6.8 demonstrates initializing a character array with a string literal, reading a string into a character array, printing a character array as a string and accessing a string's individual characters. The program uses a `for` statement (lines 20–22) to loop through the `string1` array and print the individual characters separated by spaces, using the `%c` conversion specification. The condition in the `for` statement is true while the counter is less than the array's size and the terminating null character has not been encountered in the string. This program reads only strings that do not contain whitespace characters. We'll show how to read strings containing whitespace characters in Chapter 8.

```
1 // fig06_08.c
2 // Treating character arrays as strings.
3 #include <stdio.h>
4 #define SIZE 20
```

Fig. 6.8 | Treating character arrays as strings. (Part 1 of 2.)

```c

// function main begins program execution
int main(void) {
   char string1[SIZE] = ""; // reserves 20 characters
   char string2[] = "string literal"; // reserves 15 characters

   // prompt for string from user then read it into array string1
   printf("%s", "Enter a string (no longer than 19 characters): ");
   scanf("%19s", string1); // input no more than 19 characters

   // output strings
   printf("string1 is: %s\nstring2 is: %s\n", string1, string2);
   puts("string1 with spaces between characters is:");

   // output characters until null character is reached
   for (size_t i = 0; i < SIZE && string1[i] != '\0'; ++i) {
      printf("%c ", string1[i]);
   }

   puts("");
}
```

```
Enter a string (no longer than 19 characters): Hello there
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o
```

Fig. 6.8 | Treating character arrays as strings. (Part 2 of 2.)

Self Check

1 *(Multiple Choice)* Which of the following statements a), b) or c) is *false*?

a) A character array representing a string can be output with `printf` and the `%s` conversion specifier, as in:

```c
printf("%s\n", month);
```

b) Function `printf`, like `scanf`, does not check how large the character array is.

c) When function `printf` displays the characters of a character array representing a string, it stops when it tries to print the first character past the end of the array.

d) All of the above statements are *true*.

Answer: c) is *false*. Actually, `printf` keeps displaying characters until it encounters a terminating null character, even if that is well beyond the end of the array.

2 *(True/False)* The following array can store a string of at most 20 characters and a terminating null character:

```c
char name1[20];
```

Answer: *False*. Actually, the statement creates a character array capable of storing a string of at most 19 characters and a terminating null character.